

# AI Guitar Song Retrieval Agent

## 1. Choice of Agent Platform, Model, and Architecture

This project implements a bounded AI agent system for guitar-song retrieval. The chosen use case is a chord assistant that helps a user retrieve structured chord data for songs, including title, artist, capo, and sectioned chord progressions. The problem is relevant because chord information on the web is often fragmented, noisy, and difficult to reuse in a structured way. A guitarist may find a song online, but the data is rarely normalized or easy to query across multiple songs. The system therefore combines retrieval, parsing, storage, and agent-based access in one workflow.

The chosen platform is **Hermes Agent**, which was one of the required platforms and fit the project well because it supports skills, MCP tool use, and multiple user interfaces. Hermes was used both in the terminal and through Discord DM. This was a meaningful choice because the agent itself did not need to contain all logic. Instead, Hermes acted as the orchestration layer that interpreted the user request and called tools when needed.

The model configuration used in Hermes was **GitHub Copilot with gpt-5.3-codex**. This was a pragmatic choice rather than a purely capability-driven one. Earlier model combinations were less stable or less compatible in the actual Hermes setup. For this project, a reliable supported model was more important than choosing the theoretically strongest model. This is an important reflection on cost and capability: larger or newer models may offer stronger reasoning, but operational stability matters more in a working prototype that must be demonstrated.

Architecturally, the system consists of the following layers:

- **User interface:** Hermes CLI and Discord DM
- **Agent layer:** Hermes Agent with a custom song lookup skill
- **Tool layer:** a local MCP server exposing chord retrieval as a callable tool
- **Workflow layer:** an n8n workflow (search\_cw\_webhook)
- **Retrieval/storage layer:** Neo4j for both graph RAG and vector RAG
- **External fallback source:** ChordsWorld

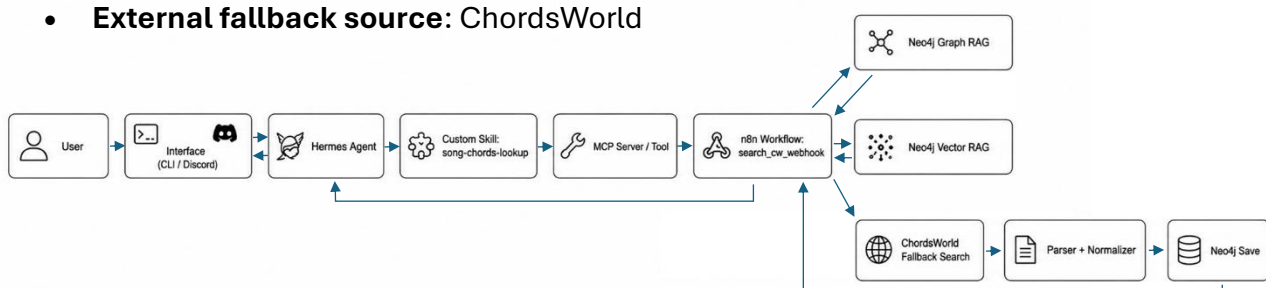


Figure 1. Overall architecture of the AI Guitar Song Retrieval Agent.

Figure 1 shows the overall system flow. The user interacts with Hermes by asking Hermes for chords through CLI or Discord, Hermes calls a custom skill and an MCP tool, and the tool triggers an n8n workflow. The workflow retrieves data from Neo4j using both graph RAG and vector RAG, and falls back to ChordsWorld when a song is missing before parsing, storing, and returning the result.

## 2. Implementation Overview

The implementation centers on the `search_cw_webhook` n8n workflow. This workflow is exposed through the local MCP server, which allows Hermes to use it as a tool instead of hard-coding the retrieval logic inside the agent.

The workflow performs the following steps:

1. Receive the user query through a webhook.
2. Normalize the query and clean noisy terms such as “chords” or “by”.
3. Query Neo4j for an existing matching song.
4. If a match exists, return the stored result directly.
5. If not, perform a fallback search against ChordsWorld.
6. Select the best candidate URL.
7. Download and parse the selected song page.
8. Extract title, artist, capo, and sectioned chord data.
9. Store the new song in Neo4j.
10. Return a formatted result back to Hermes.

This made n8n a meaningful part of the system rather than just a wrapper. It orchestrates retrieval, fallback, parsing, persistence, and formatting.

### MCP

MCP is used as the boundary between Hermes and the workflow layer. The local MCP server exposes the chord retrieval action as a tool. This is important because the agent can call the workflow in a structured way rather than relying on ad hoc scripting. In practice, MCP is what made the n8n workflow usable from Hermes and later from Discord.

## Neo4j Graph RAG

Graph RAG is implemented through structured Neo4j nodes and relationships, including:

- `(:Song)-[:BY_ARTIST]->(:Artist)`
- `(:Song)-[:HAS_SECTION]->(:Section)`

This graph structure allows the system to retrieve not only a flat text record, but a song together with its related artist and sectional structure. This is a meaningful use of graph RAG because the use case depends on relationships between entities. A song is not just a text blob; it has a performer and internal musical sections.

## Neo4j Vector RAG

Vector RAG is implemented in Neo4j over stored song-related text such as summaries and chord content. Its purpose is to support semantic retrieval rather than exact title matching only. This means the project includes two retrieval modes:

- **graph-based retrieval** for structured song relationships
- **vector-based retrieval** for semantic similarity over stored textual content

This distinction is important because the assignment required both graph RAG and vector RAG to be used meaningfully.

## Student-Created Prompt / Skill

A student-created Hermes skill, `song-chords-lookup`, was used to shape the agent's behavior. The skill instructs Hermes to treat song and chord requests as retrieval tasks, to use the MCP tool as the source of truth, and to preserve section structure instead of collapsing it into short summaries. This was important because early results showed that the model tended to compress multiple sections into shorter progression-style answers. The skill was then adjusted so the output preserved Intro, Verse, Pre-Chorus, Chorus, Bridge, and similar labels when available.

The n8n workflow also includes student-created logic choices, for example:

- query cleanup before lookup
- deterministic fallback candidate selection
- section repair for noisy parse output

- capo extraction from raw HTML and normalized text

These were important in practice because retrieval quality and parser robustness had a larger effect on user experience than the model alone.

### 3. Use of the Agent in the Chosen Case

The system was tested through Hermes CLI and later through Discord DM. This demonstrated that the same underlying tool chain could support more than one interface.

Several successful test cases were used:

- Wonderwall by Oasis
- Yellow by Coldplay
- Perfect by Ed Sheeran
- Drivers License by Olivia Rodrigo

These tests represented different types of behavior. Some songs were retrieved directly from Neo4j when already stored. Others triggered fallback ingestion, were parsed from ChordsWorld, saved into Neo4j, and then returned to the user. This showed that the system is not only a static lookup database but also an ingestion workflow.

A particularly useful case was Drivers License by Olivia Rodrigo, which demonstrated that a song not yet in Neo4j could be found through fallback, parsed, returned to the user, and then confirmed in Neo4j afterward. This is important because it proves the full end-to-end chain: Discord/Hermes → MCP → n8n → fallback source → Neo4j → response.

The project also produced a controlled failure case. When testing Locked Out of Heaven by Bruno Mars, the workflow did not return a usable final item. Instead of hallucinating a confident but false answer, the system reported that the song could not be found in the workflow-backed corpus. This is a strength in a bounded retrieval system: failing honestly is preferable to fabricating content.

### 4. Reflection, Limitations, Security, and Future Work

A major lesson from the project is that a useful agent system depends as much on workflow design and retrieval quality as on the language model itself. In this project, the hardest problems were not generic prompting, but candidate ranking, parsing, and structured formatting. For example, some songs required fallback tuning, better candidate scoring, or

section repair to produce usable output. This is a realistic insight: in applied agent systems, the surrounding tool and retrieval architecture matters greatly.

The main limitations are:

1. ChordsWorld parsing is helpful but imperfect.
2. Candidate ranking can still fail for some songs.
3. Some songs required song-specific fixes, which reduces generality.
4. The dataset is still relatively small, so coverage is limited.
5. Discord gateway setup required non-trivial configuration of token management, intents, allowlists, and pairing behavior.

There are also security and integrity considerations. The system uses external web content as fallback, so scraped data should not be treated as authoritative without validation. Bot tokens and gateway permissions must be handled carefully. During development, Discord token rotation and access control became a concrete operational concern. From an academic integrity perspective, the system is not meant to replace human judgment. It helps retrieve and structure material, but the user must still evaluate whether the result is musically correct and appropriate. Neo4j was used most concretely for graph-based retrieval in the final prototype. A vector-based retrieval layer was part of the intended architecture, but it was not implemented or demonstrated as clearly as the graph-based layer. Extending the system with explicit embedding storage and vector similarity search would therefore be a natural next step.

Future work would include:

- improving candidate ranking before ingestion
- adding stronger validation before saving scraped songs
- supporting more than one chord source
- reducing the need for song-specific parser fixes
- improving server-channel behavior in Discord
- expanding the Neo4j corpus for better graph and vector retrieval coverage

Overall, the prototype satisfies the core project requirements. It uses Hermes Agent, MCP, n8n, and Neo4j graph retrieval as real parts of the implemented solution, while a Neo4j vector-based layer remained part of the intended architecture and future work. It solves a bounded and relevant problem, works in practice, and can be demonstrated through both CLI and Discord.